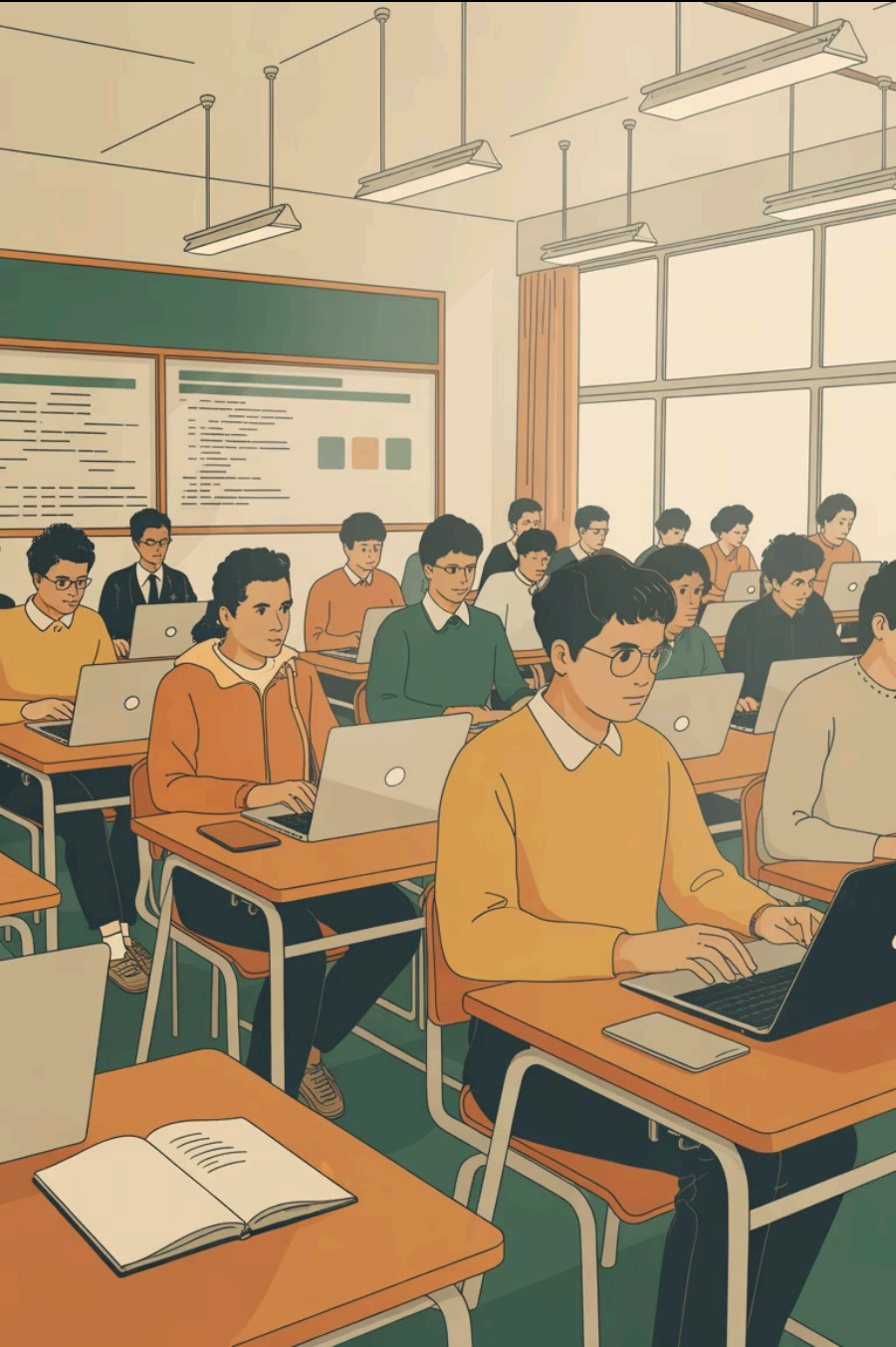




ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Classificação de Frameworks Back-end por Linguagem

Java • JavaScript • Python • PHP

Por que classificar frameworks?

Problema Inicial

"Se existem tantos frameworks, como escolher o certo?"

- Cada linguagem possui múltiplas opções
- Cada framework resolve um tipo de problema
- Escolha errada = retrabalho, baixo desempenho ou alta complexidade

Objetivos

→ Compreender

Como frameworks são classificados por tipo, arquitetura e responsabilidade

→ Comparar

Frameworks entre diferentes linguagens e seus ecossistemas

→ Entender

O papel de cada tipo de framework em uma aplicação

→ Decidir

Desenvolver visão crítica para escolha da tecnologia adequada ao contexto

O que é Classificação de Frameworks?

Frameworks podem ser classificados por quatro dimensões principais:

Tipo de Aplicação

Web, API REST, microserviços, full stack

Arquitetura

MVC, DDD, Clean Architecture, Event-Driven

Responsabilidade

Aplicação, Banco de Dados, Segurança, IoC/DI

Nível de Abstração

Micro (leve) vs Macro (robusto, opinativo)

Principais Categorias

01

Frameworks de Aplicação

Web / APIs REST

02

IoC / DI

Inversão de Controle e Injeção de Dependência

03

Banco de Dados

ORM e Query Builder

04

Segurança

Autenticação, Autorização, OAuth/JWT

05

Full Stack

Solução completa ponta a ponta

Visão Geral por Linguagem

Cada linguagem carrega uma identidade própria que molda seus frameworks

Java

Robusto e corporativo

Ecosistema maduro, forte uso de padrões de projeto, amplamente adotado em sistemas enterprise

JavaScript

Flexível e leve

Baseado em eventos, alta performance para I/O, muito usado em APIs assíncronas e tempo real

Python

Simples e produtivo

Elegância e produtividade, forte presença em APIs, prototipagem rápida e Inteligência Artificial

PHP

Tradicional web

Forte presença no web tradicional, evoluiu muito nos últimos anos com frameworks modernos



JAVA

Ecossistema Java



Aplicação / APIs

Spring Boot e Jakarta EE

Criação de APIs REST,
gerenciamento do ciclo
de vida da aplicação



IoC / DI

Spring Framework

Inversão de controle,
container de
dependências, Beans
gerenciados



Banco de Dados (ORM)

Hibernate e JPA

Mapeamento objetos →
tabelas relacionais com
abstração completa do
SQL



Segurança

Spring Security

Autenticação,
Autorização, OAuth e JWT
integrados ao
ecossistema Spring

Ecosistema JavaScript



Aplicação / APIs

Express e Fastify

Criação de APIs REST com middleware pipeline flexível e alta performance



IoC / DI

InversifyJS e Tsyringe

Gerenciamento de dependências e facilitação de testes automatizados



Banco de Dados (ORM)

Prisma, Sequelize e TypeORM

Opções modernas para mapeamento e manipulação de dados relacionais e não relacionais



Segurança

Passport.js

Estratégias flexíveis de autenticação com integração OAuth e JWT



Ecossistema Python



Aplicação / APIs

Django, Flask e FastAPI

Do full stack opinativo (Django) ao micro framework leve (Flask) e APIs modernas (FastAPI)



IoC / DI

dependency-injector

Python usa menos DI formal que Java — a injeção muitas vezes ocorre de forma implícita



Banco de Dados (ORM)

SQLAlchemy e Django ORM

Mapeamento flexível e integração nativa com o ecossistema Django



Segurança

Django Auth

Sistema completo de autenticação embutido no framework Django

Ecossistema PHP



Aplicação / APIs

Laravel e Symfony

Sintaxe expressiva e ecossistema maduro; Symfony oferece componentes reutilizáveis



IoC / DI

Container do Laravel e Symfony DI

Gerenciamento de dependências robusto e integrado ao ciclo de vida dos frameworks



Banco de Dados (ORM)

Eloquent ORM e Doctrine

Eloquent (Laravel) com sintaxe fluente; Doctrine (Symfony) com padrão Data Mapper



Segurança

Laravel Security

Autenticação pronta, proteção contra ataques comuns (CSRF, XSS, SQL Injection)

Comparação Geral entre Linguagens

Categoria	Java	JS (Node)	Python	PHP
Aplicação / API	Spring Boot, Jakarta EE	Express, Fastify	Django, Flask, FastAPI	Laravel, Symfony
IoC / DI	Forte	Médio	Fraco	Médio
ORM	Hibernate, JPA	Prisma, Sequelize	SQLAlchemy, Django ORM	Eloquent, Doctrine
Segurança	Completa	Modular	Integrada	Integrada
Perfil	Enterprise	APIs rápidas	IA e prototipagem	Web tradicional



Insight: "Não existe o melhor framework — existe o mais adequado ao contexto"

Como Escolher um Framework?

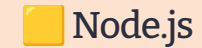
Critérios de Escolha

- 1 Tipo de projeto**
Escopo, complexidade e requisitos funcionais
- 2 Escalabilidade**
Volume de usuários e crescimento esperado
- 3 Equipe**
Curva de aprendizado e expertise atual do time
- 4 Comunidade**
Suporte, documentação e longevidade da tecnologia

Cenários Práticos



Sistemas bancários e aplicações enterprise de grande porte



APIs rápidas, escaláveis e aplicações em tempo real



Protótipos, projetos de IA e ciência de dados



Sistemas web tradicionais com ecossistemas consolidados

Erros Comuns

- Escolher pela "moda" sem avaliar requisitos
- Ignorar a expertise e tamanho da equipe
- Misturar muitos frameworks sem necessidade
- Não considerar performance e manutenibilidade

Fechamento e Discussão

Principais Conclusões

- **Frameworks resolvem problemas específicos**
Classificar ajuda a identificar a ferramenta certa para cada problema
- **Cada linguagem tem seu ecossistema**
Java, JS, Python e PHP têm identidades e pontos fortes distintos
- **Contexto é tudo**
Saber classificar frameworks é a base de uma boa tomada de decisão técnica

Perguntas para Discussão

Qual linguagem você escolheria hoje?

E qual framework usaria como ponto de partida?

Performance ou produtividade?

O que pesa mais na sua decisão de tecnologia?



Atividade

Escolha uma linguagem, liste 3 frameworks, classifique cada um e explique quando usar

OBRIGADO PELA ATENÇÃO!